**Services**

**AccountService**

-uow : UnitOfWork

+open_account(actor : User, currency_code : CurrencyCode) : Account
+get_accounts(actor : User, user_id : Optional[UUID]) : list[Account]
+get_account(actor : User, account_number : AccountNumber) : Account
+close_account(actor : User, account_number : AccountNumber) : None
+freeze_account(actor : User, account_number : AccountNumber) : None
+unfreeze_account(actor : User, account_number : AccountNumber) : None

**UserService**

-uow : UnitOfWork

+update_contact_data(actor : User, email : Email, phone_number : Phone) : None
+set_address(actor : User, address : Address) : None
+list_users(actor : User, offset : int, limit : int) : list[User]
+activate_user(actor : User, user_id : UUID) : None
+block_user(actor : User, user_id : UUID) : None
+unblock_user(actor : User, user_id : UUID) : None
+deactivate_user(actor : User, user_id : UUID) : None

**Scheduled Job Worker**

**StandingOrderTask**

-api_url : str
-service_token : str

+run() : None

HTTPS

**TransactionService**

-uow : UnitOfWork
-exchange_rates : ExchangeRateService

+transfer(actor : User, source_account : AccountNumber, target_account : AccountNumber, amount : MoneyAmount, description : str) : Transaction
+deposit(actor : User, account_number : AccountNumber, amount : MoneyAmount, description : str) : Transaction
+withdraw(actor : User, account_number : AccountNumber, amount : MoneyAmount, description : str) : Transaction
+create_standing_order(actor : User, source_account : AccountNumber, target_account : AccountNumber, amount : MoneyAmount, description : str, cron_expression : str) : ScheduledTransaction
+get_standing_orders(actor : User, account_number : AccountNumber) : list[ScheduledTransaction]
+cancel_standing_order(actor : User, standing_order_id : UUID) : None
+execute_due_standing_orders(now : datetime, limit : int) : int
+request_reversal(actor : User, transaction_id : UUID, reason : str) : Transaction
+list_reversal_requests(actor : User) : list[Transaction]
+approve_reversal_request(actor : User, request_id : UUID) : Transaction
+reject_reversal_request(actor : User, request_id : UUID) : Transaction
+reverse_transaction(actor : User, transaction_id : UUID, reason : str) : Transaction

**ReportService**

-uow : UnitOfWork
-pdf_renderer : PdfRenderer

+get_history(actor : User, account_number : AccountNumber, period_from : datetime, period_to : datetime) : list[LedgerEntry]
+print_statement(actor : User, account_number : AccountNumber, period_from : datetime, period_to : datetime) : bytes
+print_transaction_list(actor : User, period_from : datetime, period_to : datetime, account_number : Optional[AccountNumber]) : bytes

**AuthService**

-uow : UnitOfWork
-password_hasher : PasswordHasher
-token_secret : str

+register(first_name : Name, last_name : Name, email : Email, pesel : Pesel, phone_number : Phone, password : Password) : User
+login(email : Email, raw_password : str) : str
+authenticate(token : str) : User

**ExchangeRateService**

-uow : UnitOfWork

+get_current_rates() : list[ExchangeRate]
+convert(amount : MoneyAmount, source : CurrencyCode, target : CurrencyCode) : ConversionResult
+set_rate(actor : User, base : CurrencyCode, target : CurrencyCode, rate : Decimal) : ExchangeRate

**PdfRenderer**

+render_statement(statement : Statement) : bytes
+render_transaction_list(transactions : list[Transaction]) : bytes

**PasswordHasher**

+hash(password : Password) : str
+verify(raw_password : str, password_hash : str) : bool

**Data access**

**UnitOfWork**

-session : DbSession
+users : UserRepository
+accounts : AccountRepository
+transactions : TransactionRepository
+ledger : LedgerRepository
+currencies : CurrencyRepository
+exchange_rates : ExchangeRateRepository
+scheduled_transactions : ScheduledTransactionRepository

+commit() : None
+rollback() : None

**TransactionRepository**

-session : DbSession

+get(transaction_id : UUID) : Optional[Transaction]
+list_pending_reversal_requests() : list[Transaction]
+list_by_period(period_from : datetime, period_to : datetime, account_id : Optional[UUID]) : list[Transaction]
+save(transaction : Transaction) : None

**AccountRepository**

-session : DbSession

+get_by_number(account_number : AccountNumber) : Optional[Account]
+get_internal(currency_code : CurrencyCode) : Optional[Account]
+list_by_user(user_id : UUID) : list[Account]
+lock(account_ids : list[UUID]) : list[Account]
+save(account : Account) : None

**UserRepository**

-session : DbSession

+get(user_id : UUID) : Optional[User]
+get_by_email(email : Email) : Optional[User]
+list(offset : int, limit : int) : list[User]
+exists_by_email(email : Email) : bool
+exists_by_pesel(pesel : Pesel) : bool
+save(user : User) : None

**LedgerRepository**

-session : DbSession

+list_by_account(account_id : UUID, period_from : datetime, period_to : datetime) : list[LedgerEntry]
+balance_at(account_id : UUID, moment : datetime) : Decimal

**ScheduledTransactionRepository**

-session : DbSession

+get(standing_order_id : UUID) : Optional[ScheduledTransaction]
+list_by_account(account_id : UUID) : list[ScheduledTransaction]
+list_due(now : datetime, limit : int) : list[ScheduledTransaction]
+save(order : ScheduledTransaction) : None

**ExchangeRateRepository**

-session : DbSession

+get_latest(base : CurrencyCode, target : CurrencyCode) : Optional[ExchangeRate]
+list_latest() : list[ExchangeRate]
+add(rate : ExchangeRate) : None

**CurrencyRepository**

-session : DbSession

+get(code : CurrencyCode) : Optional[Currency]
+list_all() : list[Currency]

**Domain**

**ScheduledTransaction**

-id : UUID
-source_account_id : UUID
-target_account_id : UUID
-amount : MoneyAmount
-currency : CurrencyCode
-description : str
-cron_expression : str
-next_run_at : datetime
-status : ScheduledStatus
-created_at : datetime

+create(source_account_id : UUID, target_account_id : UUID, amount : MoneyAmount, currency : CurrencyCode, description : str, cron_expression : str) : ScheduledTransaction
+is_due(now : datetime) : bool
+advance(now : datetime) : None
+cancel() : None

0..* 0..* 0..1 generates 0..*

**Transaction**

-id : UUID
-reference_number : str
-description : str
-status : TransactionStatus
-executed_at : datetime
-reverses_transaction_id : Optional[UUID]
-scheduled_transaction_id : Optional[UUID]
-initiated_by : Optional[UUID]
-entries : list[LedgerEntry]

+create(reference_number : str, description : str, initiated_by : Optional[UUID], scheduled_transaction_id : Optional[UUID]) : Transaction
+create_reversal_request(reference_number : str, original : Transaction, reason : str, requested_by : UUID) : Transaction
+debit(account : Account, amount : MoneyAmount) : None
+credit(account : Account, amount : MoneyAmount) : None
+apply_reversal_of(original : Transaction) : None
+post() : None
+reject() : None
+mark_reversed() : None
+is_reversible() : bool

0..1 reverses 0..1

1 target source 1

0..* initiated_by 0..1

**User**

-id : UUID
-client_number : ClientNumber
-pesel : Pesel
-first_name : Name
-last_name : Name
-email : Email
-phone_number : Phone
-password_hash : str
-role : Role
-status : UserStatus
-created_at : datetime
-addresses : list[Address]

+create(client_number : ClientNumber, pesel : Pesel, first_name : Name, last_name : Name, email : Email, phone_number : Phone, password_hash : str, role : Role) : User
+is_active() : bool
+has_role(role : Role) : bool
+activate() : None
+block() : None
+unblock() : None
+deactivate() : None
+update_contact(email : Email, phone_number : Phone) : None
+set_address(address : Address) : None
+masked_pesel() : str

entries 1 1 0..* owner 0..1 addresses 0..2

**Account**

-id : UUID
-user_id : Optional[UUID]
-account_number : AccountNumber
-currency_code : CurrencyCode
-balance : Decimal {readOnly}
-account_type : AccountType
-status : AccountStatus
-created_at : datetime

+create_customer_account(user_id : UUID, account_number : AccountNumber, currency_code : CurrencyCode) : Account
+is_active() : bool
+is_owned_by(user_id : UUID) : bool
+has_sufficient_funds(amount : MoneyAmount) : bool
+can_be_closed() : bool
+freeze() : None
+unfreeze() : None
+close() : None

**Address**

-id : UUID
-address_type : AddressType
-street : Street
-building_number : AddressNumber
-apartment_number : Optional[AddressNumber]
-postal_code : Optional[PostalCode]
-city : City
-country : CountryCode
-created_at : datetime

+formatted() : str

**ExchangeRate**

-id : int
-base_currency : CurrencyCode
-target_currency : CurrencyCode
-rate : Decimal
-rate_timestamp : datetime

+convert(amount : MoneyAmount) : Decimal
+inverse() : ExchangeRate

0..* 0..* base target 1 1

1 1 1 1

**Result Objects**

**ConversionResult**

-source_amount : MoneyAmount
-source_currency : CurrencyCode
-target_amount : Decimal
-target_currency : CurrencyCode
-rate : Decimal

**Statement**

-account_number : AccountNumber
-currency_code : CurrencyCode
-period_from : datetime
-period_to : datetime
-opening_balance : Decimal
-closing_balance : Decimal
-entries : list[LedgerEntry]

entries 0..* 0..* 0..*

**LedgerEntry**

-id : UUID
-transaction_id : UUID
-account_id : UUID
-entry_type : EntryType
-amount : MoneyAmount
-currency_code : CurrencyCode
-created_at : datetime

+signed_amount() : Decimal
+inverted(transaction_id : UUID) : LedgerEntry

**Currency**

-code : CurrencyCode
-name : str
-symbol : CurrencySymbol

**Enums**

«enumeration» UserStatus: ACTIVE, PENDING, BLOCKED, DELETED

«enumeration» Role: CUSTOMER, ADMIN

«enumeration» AddressType: RESIDENTIAL, CORRESPONDENCE

«enumeration» AccountType: CUSTOMER, INTERNAL

«enumeration» AccountStatus: ACTIVE, FROZEN, CLOSED

«enumeration» TransactionStatus: PENDING, POSTED, REVERSED, REJECTED

«enumeration» EntryType: DEBIT, CREDIT

«enumeration» ScheduledStatus: ACTIVE, CANCELLED